KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY
Department of Computer Science & Engineering

OOP LAB PROJECT REPORT

Digital Logic Simulator

A GUI-Based Boolean Expression Evaluator with Circuit Visualization

---

Course Name	Object Oriented Programming
Course Code	CSE 1206
Submitted By	Turjo Deb
Roll No.	2407084
Year	1st Year
Session	2nd Session
Department	Computer Science & Engineering
University	Khulna University of Engineering & Technology

Submitted To
Md Mehrab Hossain Opi & Safin Ahmmed
Lecturers, Department of Computer Science & Engineering
Khulna University of Engineering & Technology

1. Introduction

1.1 Project Overview

The Digital Logic Simulator is a GUI-based desktop application developed in C++ using the SFML graphics library. It allows users to type any boolean expression using standard logical operators and instantly visualize the corresponding logic circuit diagram alongside its complete truth table. The application supports AND, OR, NOT, and XOR gates with parentheses and operator precedence, making it suitable for evaluating complex multi-gate expressions.

1.2 Problem Statement

Analyzing digital logic circuits manually is time-consuming and error-prone, especially for expressions involving many variables. Students and engineers often need to construct truth tables and trace signal paths to verify circuit behavior. This project automates that entire workflow: a user provides a boolean expression as text input, and the simulator parses it, builds an expression tree, evaluates all possible input combinations, and renders both a visual circuit and a truth table in real time.

1.3 Objectives

- Apply core OOP principles — classes, inheritance, polymorphism, encapsulation, and abstraction — in a real C++ project.
- Implement a recursive descent parser to transform boolean expressions into a runtime gate tree.
- Render logic gate diagrams (AND, OR, NOT, XOR) graphically using SFML, with signal-colored wire highlighting.
- Generate and display a complete truth table (2^n rows) for any valid boolean expression.
- Demonstrate generic programming using C++ templates and operator overloading.
- Provide file export functionality to save truth table results as a .txt file.

2. System Features

The simulator provides the following user-facing features:

- Boolean Expression Input — Users type expressions using operators & (AND), | (OR), ! (NOT), ^ (XOR), and parentheses. Operator precedence is respected: NOT > AND > XOR > OR.
- Automatic Truth Table Generation — For any n-variable expression, the system enumerates all 2^n input combinations and evaluates each row, displaying the result in a scrollable right panel.

- Visual Circuit Diagram — A graphical representation of the gate tree is rendered in the left panel, showing gate shapes (D-shape for AND, curved for OR, triangle for NOT, curved-with-curve for XOR) and interconnecting wires.
- Row Selection and Wire Highlighting — Clicking any row in the truth table causes the circuit wires to be recolored: green for signal=1, red for signal=0, allowing visual signal tracing.
- Expression Export — The EXPORT button writes the current truth table (with expression header and variable columns) to a .txt file on disk using std::ofstream.
- Error Handling — Invalid expressions trigger a std::runtime_error that is caught and displayed as a status message at the bottom of the window.
- Resizable Window — The layout recalculates on window resize, keeping panels and controls proportional.

3. System Specifications

Specification	Detail
Language	C++17
Graphics Library	SFML 2.x (Simple and Fast Multimedia Library)
Development Environment	VS Code / g++ compiler
Operating System	Windows / Linux
Compiler Standard	C++17 (structured bindings, shared_ptr, etc.)
Project Files	main.cpp, Gate.h, Tokenizer.h/cpp, Parser.h/cpp, TruthTable.h/cpp, CircuitDrawer.h/cpp, GUI.h/cpp
Total Lines of Code	~1030 lines across all source files
Font Asset	DejaVuSans.ttf (bundled in assets/fonts/)

4. OOP Features and Technical Implementation

OOP Concept	Status	Implementation in Project
Classes & Objects	✓Used	Node, InputNode, AndGate, OrGate, NotGate, XorGate, Parser, Tokenizer, TruthTable, GUI, CircuitDrawer
Encapsulation	✓Used	Private members: toks, pos (Parser); gates, wires (CircuitDrawer); all GUI internals
Constructors & Destructors	✓Used	GUI(), InputNode(string,bool), AndGate(ptr,ptr); virtual ~Node() in base class
Inheritance	✓Used	5 gate classes (InputNode, AndGate, OrGate, NotGate, XorGate) inherit from Node
Dynamic Polymorphism	✓Used	virtual evaluate()=0 in Node; overridden in all 5 subclasses — runtime dispatch
Static Polymorphism	✓Used	Row::operator== and Row::operator<< overloaded in TruthTable::Row struct
Abstraction	✓Used	Node is pure abstract class; .h/.cpp header separation hides implementation
File Handling	✓Used	std::ofstream in GUI::onExportClicked() writes truth table to .txt file
STL Containers	✓Used	vector<Row>, map<string,bool>, vector<string>, vector<sf::Vertex>
Exception Handling	✓Used	try-catch block in GUI.cpp; throw std::runtime_error in Parser.cpp
Static Members	✓Used	static const color constants (COL_WIRE, COL_HIGH, COL_LOW) and PI in CircuitDrawer.cpp
Templates	✓Used	printCell<T>() in TruthTable.h; instantiated as <string> and <bool>
Memory Management	✓Used	std::shared_ptr<Node> used for entire expression tree — automatic memory cleanup

4.1 Classes and Objects

The project is structured around 13 classes and structs, each with a clearly defined responsibility. The central abstraction is the Node base class from which all gate types derive.

Class / Struct	File	Role
Node (abstract)	Gate.h	Base class — pure virtual evaluate()
InputNode	Gate.h	Leaf node — holds boolean variable value
AndGate	Gate.h	AND logic: left && right
OrGate	Gate.h	OR logic: left || right
NotGate	Gate.h	NOT logic: !child
XorGate	Gate.h	XOR logic: left ^ right

Class / Struct	File	Role
Tokenizer	Tokenizer.h/cpp	Splits expression string into token list
Parser	Parser.h/cpp	Recursive descent parser — builds Node tree
TruthTable	TruthTable.h/cpp	Enumerates 2^n rows; stores and prints results
TruthTable::Row	TruthTable.h	Struct with operator== and operator<< overloads
CircuitDrawer	CircuitDrawer.h/cpp	Renders gate shapes and signal-colored wires via SFML
GUI	GUI.h/cpp	Main application — handles events, layout, user interaction

4.2 Encapsulation

All classes protect their internal state using private data members with controlled access through public methods. For example, the Parser class keeps its token list (toks) and current position (pos) private, exposing only the parse() method. The CircuitDrawer class hides its gates vector and wires array, exposing only build(), highlight(), and draw(). The GUI class encapsulates all SFML window objects, layout logic, and state variables privately.

4.3 Constructors and Destructors

Every class uses constructors for proper initialization. The GUI constructor sets up the SFML window, loads the font, and calls setupUI() to initialize all layout elements. Gate constructors accept shared_ptr arguments for their child nodes. The Node base class declares a virtual destructor to ensure correct cleanup when deleting derived objects through base pointers.

```
GUI::GUI() : window(sf::VideoMode(1100,680), "Digital Logic
Simulator"), inputFocused(false), hasResult(false) { ... }
virtual ~Node() {}  // in Node base class
```

4.4 Inheritance

All five gate types inherit publicly from the abstract Node base class. This creates a clean class hierarchy where each subclass provides a specific implementation of evaluate(). The inheritance chain allows the entire circuit tree to be stored as shared_ptr<Node>, regardless of actual gate type.

```
class Node { public: virtual bool evaluate() = 0; virtual ~Node() {}
};
class AndGate : public Node { ... bool evaluate() override { return
left->evaluate() && right->evaluate(); } };
```

4.5 Polymorphism

Dynamic Polymorphism: The evaluate() method is declared as a pure virtual function in Node. Each subclass overrides it with its specific logic. When the truth table calls root->evaluate(), C++ automatically dispatches to the correct gate's implementation at runtime — without any if/switch statements. This is the core mechanism that makes the recursive expression tree work.

Static Polymorphism (Operator Overloading): The TruthTable::Row struct overloads two operators. operator== allows two Row objects to be compared by value (checking both input map and output). operator<< enables streaming a Row directly to std::cout, providing clean, readable output.

```
bool operator==(const Row& other) const { return inputs ==
other.inputs && output == other.output; }
friend std::ostream& operator<<(std::ostream& os, const Row& r) { ...
}
```

4.6 Abstraction

The Node class is a pure abstract class — it cannot be instantiated directly and exists solely to define the interface all gates must implement. Implementation details of each gate are hidden behind this interface. Additionally, every class separates its interface (.h header) from its implementation (.cpp file), keeping the public API clean and hiding internal logic.

4.7 File Handling

The GUI::onExportClicked() method uses std::ofstream to write the truth table to a .txt file. The export includes the expression string, variable names, row count, and formatted truth table data. The file is opened in default write mode (overwrite), and the status bar confirms success or failure.

```
#include <fstream>
std::ofstream file(fname);
file << "Expression: " << inputString << "\n";
```

4.8 Static Members

CircuitDrawer.cpp defines several file-scope static constants used for wire and gate rendering colors. These include COL_WIRE (default wire color), COL_HIGH (green for signal=1), COL_LOW (red for signal=0), COL_GATE, COL_INPUT, and mathematical constant PI. Being static, they are shared across all instances and computed once at program startup.

```
static const float PI = 3.14159265f;
static const sf::Color COL_HIGH = sf::Color(80, 220, 100);  //
signal=1 green
static const sf::Color COL_LOW  = sf::Color(220, 80, 80);   //
signal=0 red
```

4.9 Templates

A generic printCell<T>() function template is defined in TruthTable.h. It accepts any type T and prints the value with a specified column width using std::setw. The function is instantiated twice in TruthTable::print(): once with T=std::string to print variable name headers, and once with T=bool to print cell values. This demonstrates how a single generic function can work across different data types without code duplication.

```
template<typename T>
void printCell(T val, int width = 4) {
    std::cout << std::setw(width) << val;
}
// Usage:
printCell<std::string>(v);       // prints header names
printCell<bool>(row.inputs.at(v)); // prints 0/1 values
```

4.10 STL Containers and Algorithms

The project makes heavy use of the C++ Standard Template Library. std::vector is used for token lists, gate collections, wire vertex arrays, and text element lists. std::map is used to store the variable-to-InputNode mapping in the Parser, and the input values in TruthTable::Row. Since std::map sorts keys alphabetically by default, variable names (A, B, C, D) are always presented in order without an explicit sort call.

4.11 Exception Handling

The Parser throws std::runtime_error for two error conditions: an unexpected token encountered during parsing, and a missing closing parenthesis. In GUI::onRunClicked(), a try-catch block wraps the tokenize-parse-generate pipeline. If any exception is thrown, the catch block updates the status bar text with the error message, preventing a crash and providing user feedback.

```
try { /* tokenize + parse + generate */ }
catch (std::exception& e) { statusText.setString("Error: " +
std::string(e.what())); }
```

4.12 Memory Management

The entire expression tree uses std::shared_ptr<Node> for all node pointers. When the Parser creates gate objects (make_shared<AndGate>(...)), the shared pointer's reference count manages the lifetime automatically. When a new expression is parsed, the old tree root is simply replaced and the old objects are freed. There is no manual new/delete in the codebase, eliminating memory leaks and dangling pointer risks.

5. System Design (UML)

5.1 Program Flowchart

The flowchart below illustrates the complete program flow from application launch through expression evaluation, circuit rendering, and optional export.

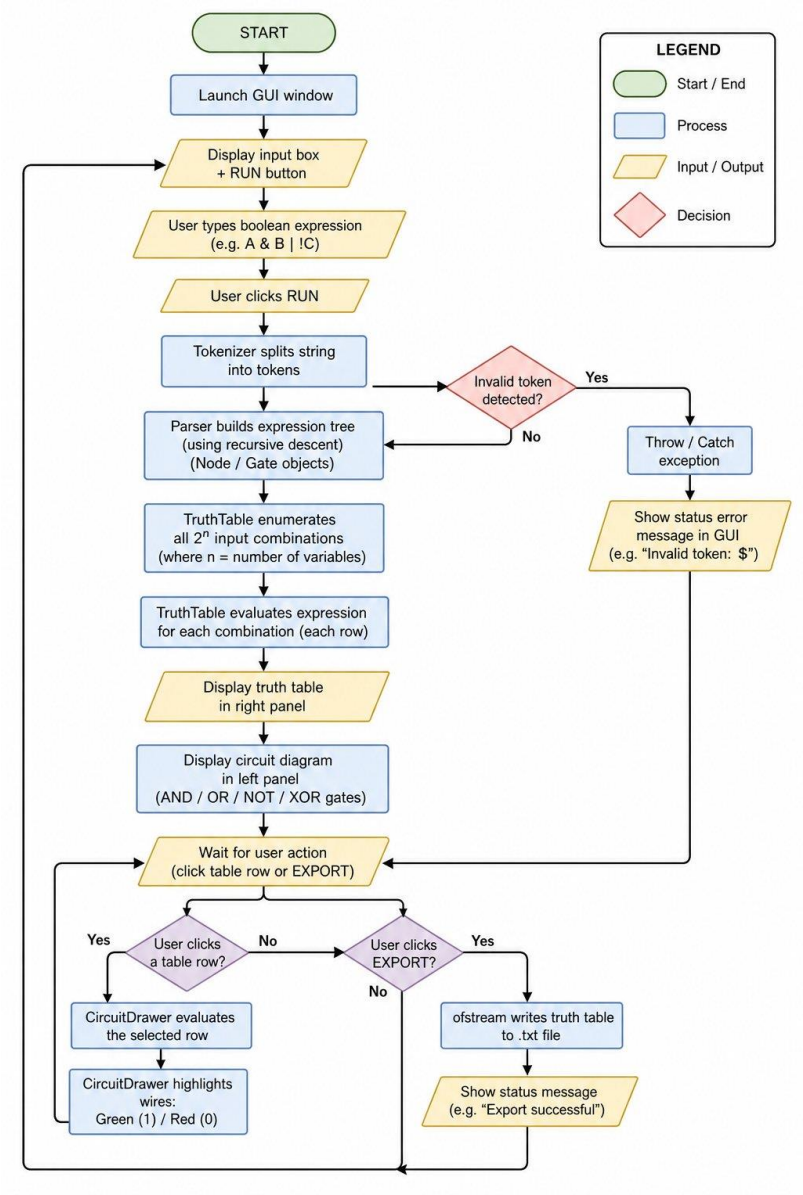


Figure: Program Flowchart — Digital Logic Simulator

5.2 Class Hierarchy

The inheritance hierarchy centers on the abstract Node base class. All gate types extend Node and override the evaluate() method. The GUI class acts as the top-level controller, coordinating the Tokenizer, Parser, TruthTable, and CircuitDrawer.

`GUI → Tokenizer → Parser → TruthTable → CircuitDrawer`

6. Key Logic and Implementation

6.1 Recursive Descent Parser

The Parser uses a classic recursive descent technique to handle operator precedence. It defines one function per precedence level, calling the next higher level first. The call chain is: parseOr() → parseXor() → parseAnd() → parseNot() → parsePrimary(). This ensures that NOT binds tightest, followed by AND, XOR, and finally OR. The result is a tree of shared_ptr<Node> objects that mirrors the structure of the expression.

6.2 Truth Table Generation

TruthTable::generate() enumerates all 2^n input combinations using a single integer loop from 0 to 2^n - 1. For each integer i, individual bits are extracted using bit-shift operations and assigned to the corresponding InputNode values. The expression tree is then evaluated by calling root->evaluate(), which recursively traverses the tree and returns the boolean result. Each row is stored as a TruthTable::Row struct containing the input map and output value.

```
int totalRows = 1 << n;  // 2^n
bool val = (i >> (n - 1 - j)) & 1;  // extract bit j
```

6.3 Circuit Rendering

CircuitDrawer::build() traverses the expression tree recursively, placing each gate at a calculated (x, y) position based on tree depth and leaf count. Gate shapes are drawn using SFML ConvexShape and CircleShape primitives. Wires between gates are rendered as quadratic Bezier curves using a custom bezier2() helper. The layout algorithm distributes gates vertically based on the number of leaves under each subtree, preventing overlap.

6.4 Signal Wire Highlighting

When a user clicks a truth table row, CircuitDrawer::highlight() is called with the selected row's input values. The function re-traverses the tree, evaluating each node with the given inputs and coloring each wire segment green (COL_HIGH, signal=1) or red (COL_LOW, signal=0). This provides an intuitive visual trace of how signals propagate through the circuit.

7. Testing and Results

Test Case	Expression	Input	Expected Output	Result
TC-1	(A & !B) | (!A & B)	2 vars, 4 rows	XOR equivalent — 1 when inputs differ	✓Pass
TC-2	(A& !B) | (!A & B) & !(C^ D)	4 vars, 16 rows	Complex multi-gate expression evaluated	✓Pass

Test Case	Expression	Input	Expected Output	Result
			correctly	
TC-3	Row click — wire highlight	Click row 8: A=0,B=1,C=1,D=1	Green wires (signal=1), Red wires (signal=0)	✓Pass
TC-4	Export to .txt	Click EXPORT after TC-2	File saved with header + 16-row truth table	✓Pass
TC-5 (Invalid)	A&&&B	Invalid token sequence	Error: Unexpected token '&' shown in status bar	✓Pass

7.1 Main Window — Application Startup

On launch, the application displays a clean interface with an input box, RUN and EXPORT buttons, and two empty panels labeled Circuit Diagram and Truth Table. The status bar at the bottom prompts the user to enter a boolean expression.

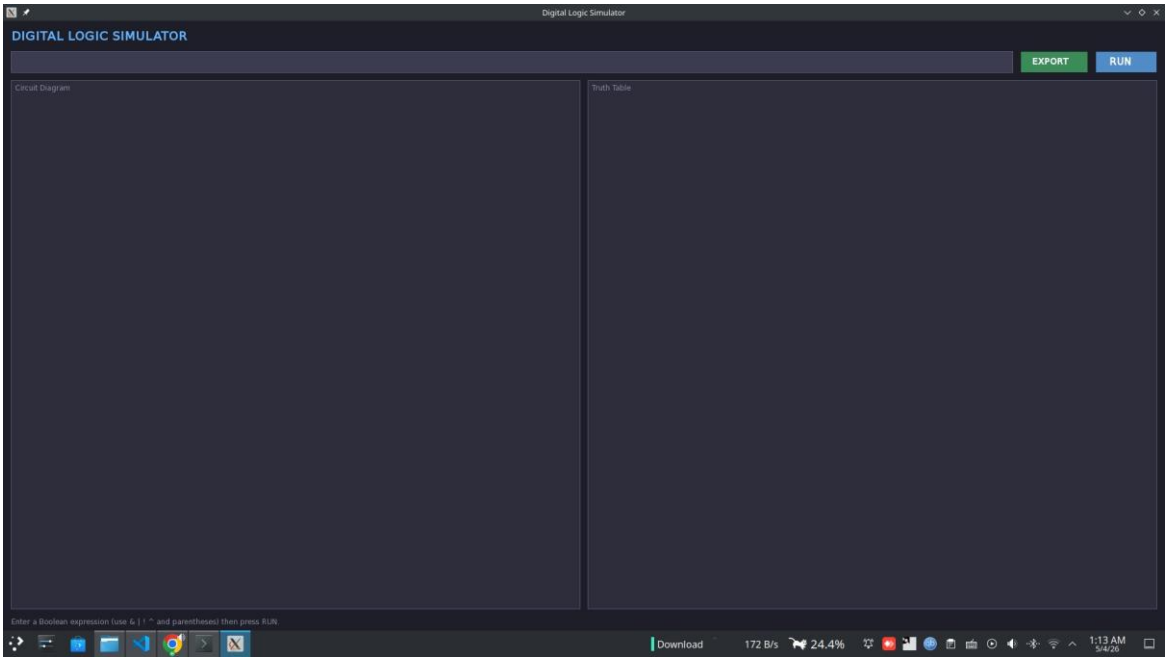


Figure: Main Window — Application Startup

7.2 Simple Expression — (A & !B) | (!A & B)

A two-variable expression (equivalent to XOR) is entered and evaluated. The circuit panel renders AND gates, NOT gates, and an OR gate. The truth table on the right shows 4 rows with the correct outputs: 0, 1, 1, 0. The status bar confirms: Vars: 2 | Rows: 4.

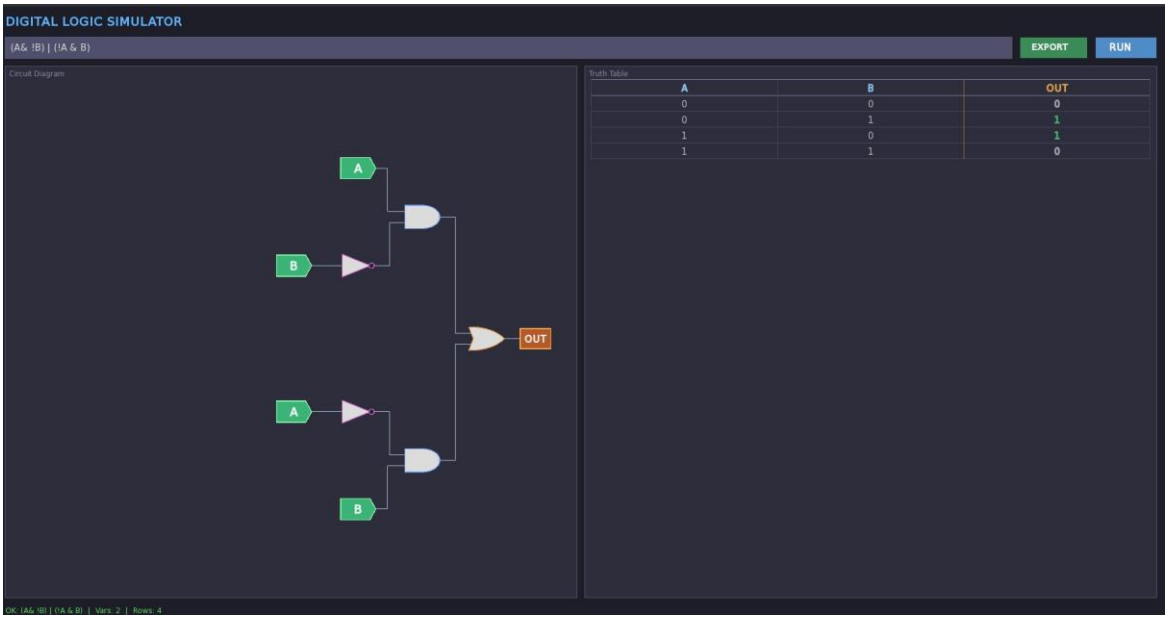


Figure: Simple Expression — (A& !B) | (!A & B)

7.3 Complex Expression — (A& !B) | (!A & B) & !(C^ D)

A four-variable expression with 16 rows is evaluated. The circuit diagram shows multiple nested gates across the left panel. The truth table correctly enumerates all 16 input combinations and their outputs. Status bar shows: Vars: 4 | Rows: 16.

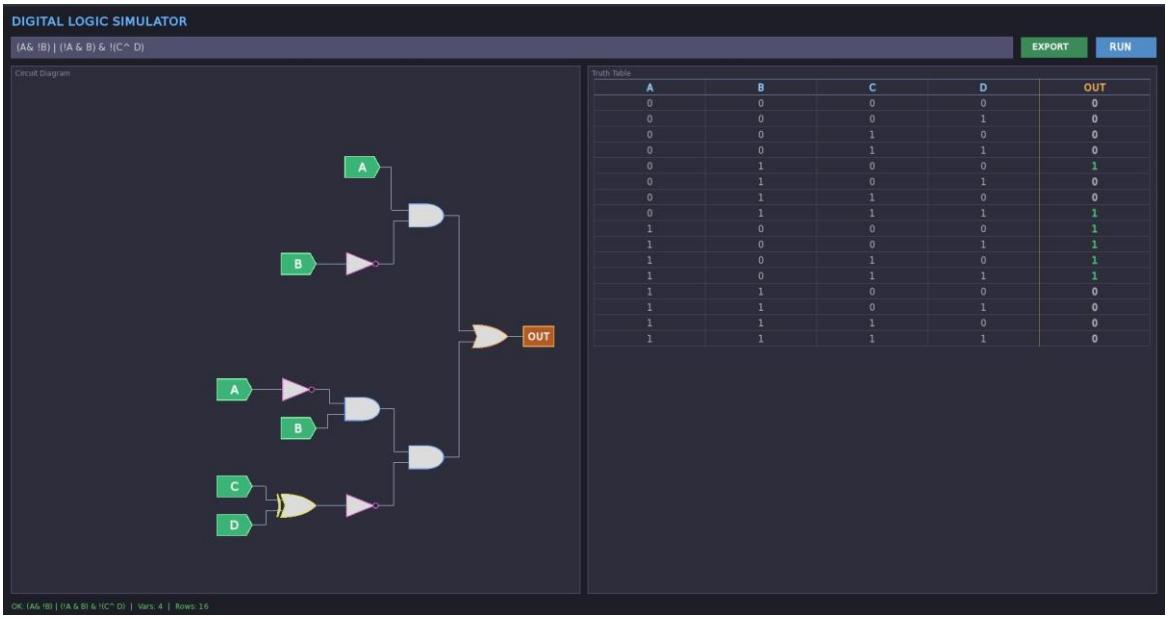


Figure: Complex Expression — 4 Variables, 16 Rows

7.4 Row Selection and Wire Highlighting

Row 8 (A=0, B=1, C=1, D=1) is selected by clicking. The circuit wires change color: green wires carry signal=1, light/neutral wires carry signal=0. The status bar displays the selected row values. This confirms correct signal propagation tracing.

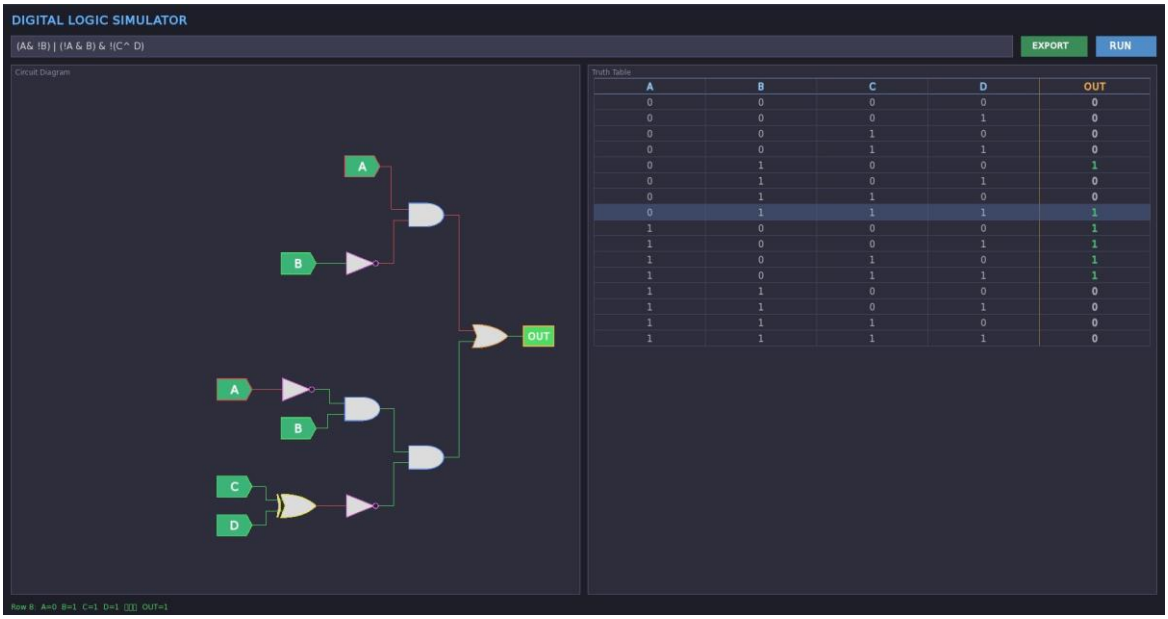


Figure: Row Selection — Wire Signal Highlighting (Green=1)

7.5 Export to Text File

Clicking EXPORT saves the truth table to a .txt file. The exported file includes the expression header, variable names, row count, separator lines, and all 16 data rows with their output values. This confirms the file handling (std::ofstream) functionality works correctly.

```
========================================
  DIGITAL LOGIC SIMULATOR - Truth Table
========================================
  Expression: (A& !B) | (!A & B) & !(C^ D)
  Variables : A B C D
  Rows      : 16
========================================

    A     B     C     D | OUT
--------------------------------
    0     0     0     0 | 0
    0     0     0     1 | 0
    0     0     1     0 | 0
    0     0     1     1 | 0
    0     1     0     0 | 1
    0     1     0     1 | 0
    0     1     1     0 | 0
    0     1     1     1 | 1
    1     0     0     0 | 1
    1     0     0     1 | 1
    1     0     1     0 | 1
    1     0     1     1 | 1
    1     1     0     0 | 0
    1     1     0     1 | 0
    1     1     1     0 | 0
    1     1     1     1 | 0
```

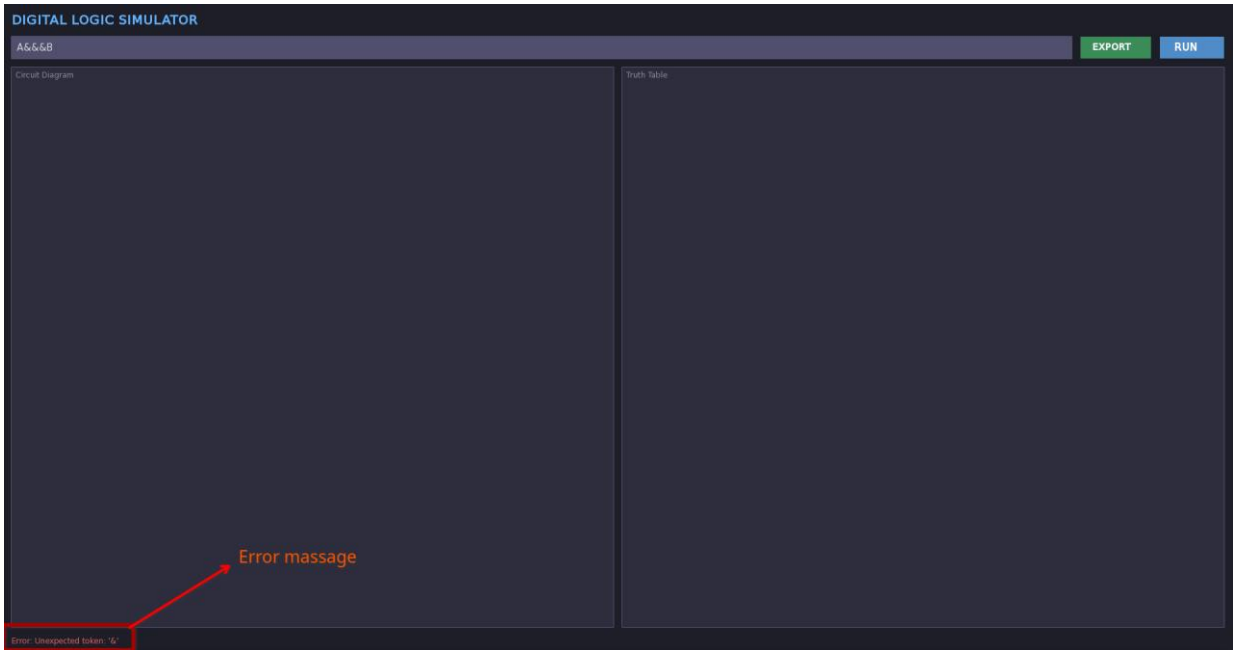
Figure: Exported Truth Table — .txt File Output

7.6 Invalid Expression — Error Handling

When an invalid expression such as 'A&&&B' is entered and RUN is clicked, the Parser throws a std::runtime_error with the message 'Unexpected token: &'. The GUI's catch block intercepts

this and displays the error message in the status bar at the bottom of the window. The circuit and truth table panels remain empty, preventing any crash or undefined behavior.

Input tested: A&&&B — Error displayed: 'Error: Unexpected token: &' (visible in status bar at bottom-left of window)



8. Challenges and Debugging

8.1 Parser Precedence Bugs

Initially, the recursive descent parser did not handle operator precedence correctly — expressions like A | B & C were evaluated left to right instead of respecting AND's higher precedence. This was fixed by restructuring the parsing functions into a strict precedence chain: parseOr → parseXor → parseAnd → parseNot → parsePrimary, where each level only calls the next higher-priority level.

8.2 Circuit Layout Overlap

Early versions of the CircuitDrawer placed gates at fixed vertical intervals, causing overlapping wires when expressions had unbalanced subtrees. This was resolved by implementing a recursive treeLeaves() function that counts the number of leaf nodes in each subtree and distributes vertical space proportionally.

8.3 SFML Font Loading

The application failed to load the font on some systems where the font was not present at the default path. A fallback path (/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf) was added

alongside the bundled asset, and the font is now included in the assets/fonts/ directory of the project to ensure consistent behavior across platforms.

8.4 Shared Pointer Tree Rebuilding

When a new expression was parsed, the old expression tree was not always fully released. This was resolved by ensuring that the root shared_ptr variable in the GUI is always replaced (not just reassigned in a local scope), which triggers automatic reference count decrement and correct cleanup of old gate objects.

9. Limitations

- File export uses plain write mode — there is no ios::app or ios::binary support. Each export overwrites the previous file rather than appending.
- Input is text-only — there is no drag-and-drop or visual gate builder. Users must know the boolean expression syntax.
- No session persistence — parsed expressions and results are lost when the application is closed. There is no save/load functionality for expressions.
- Limited gate types — NAND, NOR, XNOR, and buffer gates are not implemented. Only AND, OR, NOT, and XOR are supported.
- No expression simplification — the tool evaluates expressions as-is without simplifying to minimal sum-of-products or other canonical forms.
- Truth table size — for expressions with many variables (n > 10), the 2^n row count may cause performance slowdown.

10. Conclusion

The Digital Logic Simulator successfully demonstrates the practical application of Object-Oriented Programming principles in building a real, functional C++ application. Inheritance and dynamic polymorphism through the Node class hierarchy allowed the circuit evaluation logic to be written without any type-checking or conditional branching — adding a new gate type requires only a new subclass. Encapsulation kept each component's internal state protected, and abstraction through pure virtual interfaces kept the design modular and extensible.

Templates provided generic utility through the printCell<T> function, and operator overloading added expressive comparison and streaming capability to the TruthTable::Row struct. Exception handling through try-catch ensured robust error reporting without application crashes. File handling via std::ofstream enabled a practical export feature, and STL containers (vector, map) formed the backbone of data management throughout the project.

Future improvements could include a graphical gate builder allowing drag-and-drop circuit construction, support for NAND/NOR/XNOR gates, boolean expression simplification using

Karnaugh maps, a database or file-based expression history, and export to image formats (PNG/PDF) for documentation purposes.

11. References

- C++ Reference — https://cplusplus.com
- SFML Documentation — https://sfml-dev.org/documentation
- Stroustrup, B. — The C++ Programming Language, 4th Edition
- Recursive Descent Parsing — https://en.wikipedia.org/wiki/Recursive_descent_parser
- Shared Pointers in C++ — https://en.cppreference.com/w/cpp/memory/shared_ptr
- Digital Logic Design — Morris Mano, Digital Design, 5th Edition